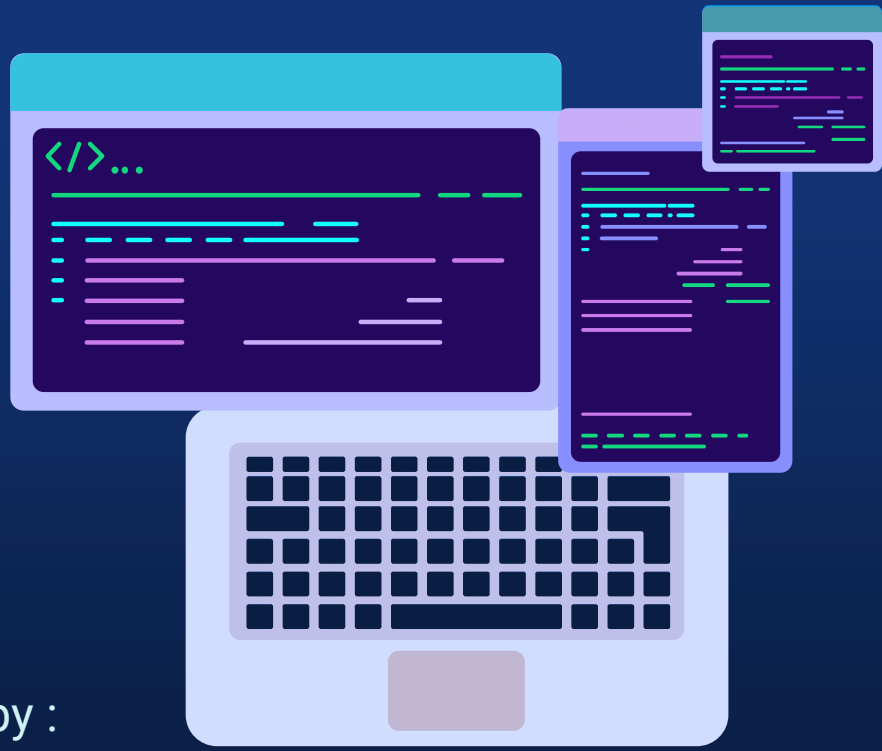
Conquering Fashion MNIST with CNNs using Computer Vision



Presented by :
Piyush Pankaj Panigrahi
Team-AlphByte
MIT Manipal(MAHE)

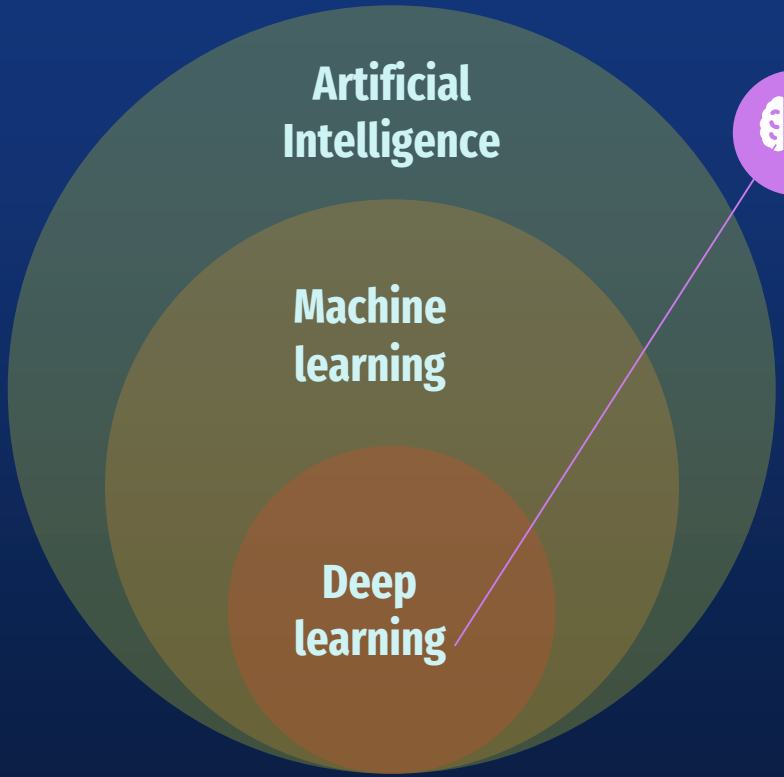
PROBLEM STATEMENT

Developing an Efficient Clothing Classification Model using Convolutional Neural Networks (CNN) and Intel Optimization.

Our main objectives in this project are :

- Our goal is to **create and train a CNN model** that can accurately categorize photos of apparel from the **Fashion MNIST** dataset. CNNs are frequently employed in computer vision tasks and have excelled in classifying images. We can accurately classify photos by capturing relevant characteristics and patterns in the images using the capabilities of CNNs.

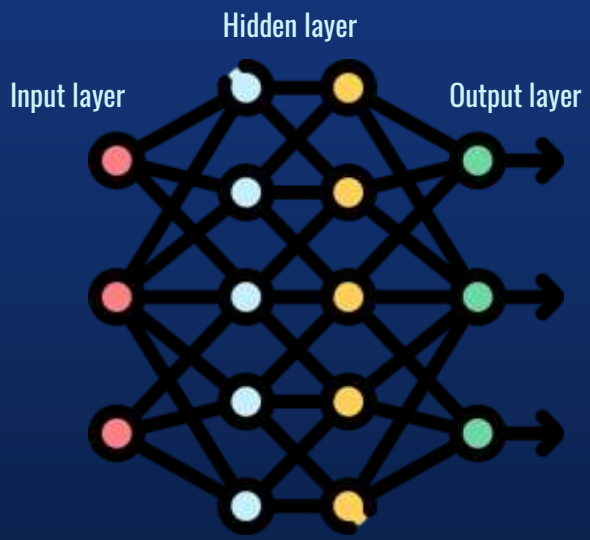
- Intel offers a variety of optimization tools and libraries that help improve the performance of deep learning models on Intel architectures. In order to speed up the inference speed of our trained CNN model, we will investigate Intel optimization approaches such as **Intel extension for Pytorch (IPEX) and Intel Distribution for Python (Intel Python)**. We can speed up predictions and increase efficiency by exploiting Intel's optimization resources, making the model appropriate for real-time applications..



Deep learning is a method in **artificial intelligence (AI)** that teaches computers to process data in a way that is inspired by the human brain. Deep learning models can recognize complex patterns in pictures, text, sounds, and other data to produce accurate insights and predictions.

In this model we use Deep Learning to classify a set of images of clothes present in **Fashion MNIST** Dataset to their original categories.

WHAT IS A CNN?

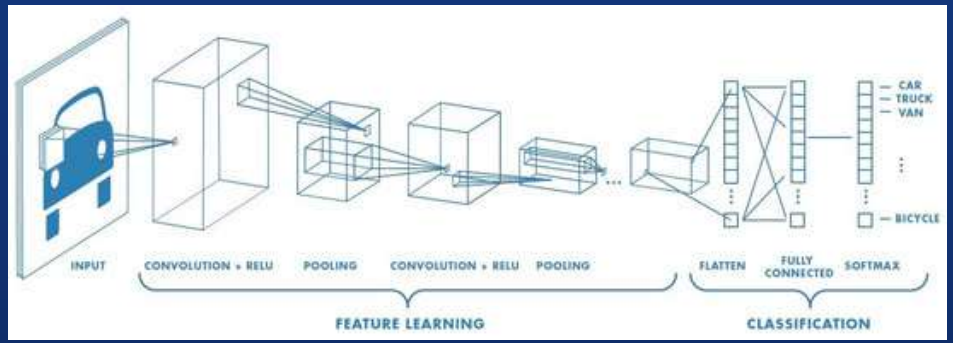


Neural Network is a type of machine learning process, called deep learning, that uses interconnected nodes or neurons in a layered structure that resembles the human brain. It is of three types :

- Artificial Neural Network (ANN)
- Convolutional Neural Network (CNN)
- Recurrent Neural Network (RNN)

A **Convolutional Neural Network**, also known as CNN or ConvNet, is a class of neural networks that specializes in processing data that has a grid-like topology, such as an image.

THE CNN ARCHITECTURE

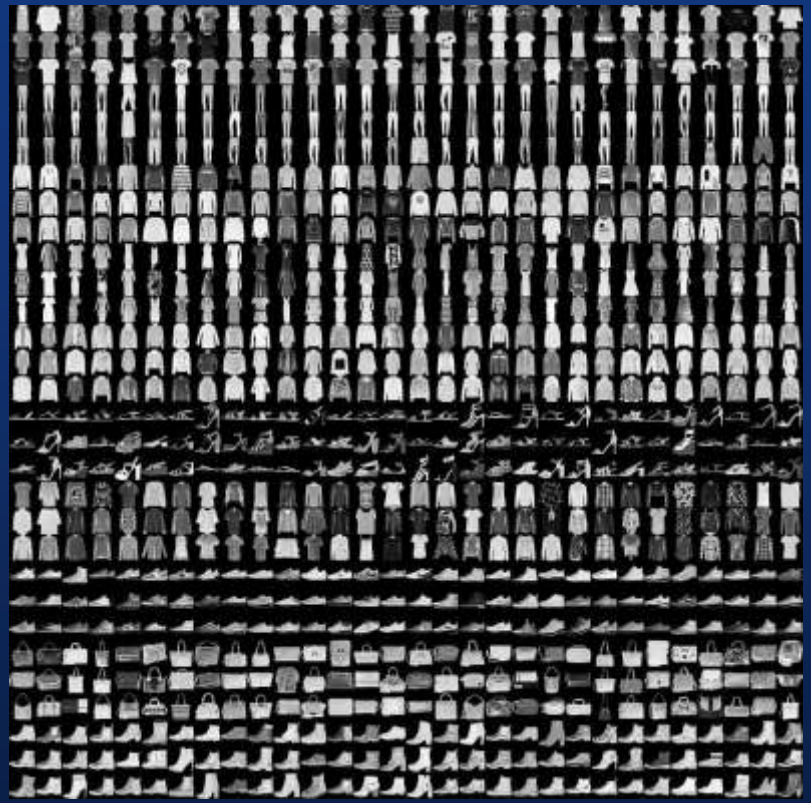


The CNNs have mainly three layers :

- Convolutional Layer
- Pooling layer
- Fully Connected (FC) Layer

The convolutional layer is the first layer of a convolutional network. While convolutional layers can be followed by additional convolutional layers or pooling layers, the fully-connected layer is the final layer.

With each layer, the CNN increases in its complexity, identifying greater portions of the image. Earlier layers focus on simple features, such as colors and edges. As the image data progresses through the layers of the CNN, it starts to recognize larger elements or shapes of the object until it finally identifies the intended object.



THE FASHION MNIST DATASET

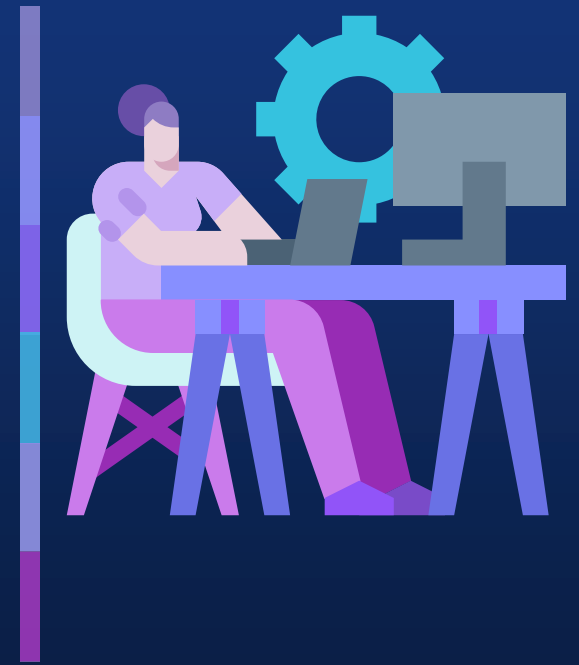
Fashion-MNIST is a dataset of Zalando's article images—consisting of a training set of 60,000 examples and a test set of 10,000 examples. Each example is a 28x28 grayscale image, associated with a label from 10 classes.

- 0 T-Shirt/Top
- 1 Trouser
- 2 Pullover
- 3 Dress
- 4 Coat
- 5 Sandal
- 6 Shirt
- 7 Sneaker
- 8 Bag
- 9 Ankle Boot

MY APPROACH TOWARDS THE PROBLEM

01 **Tensorflow** framework is used along with **Keras** API in this project for classifying the images in the dataset. Along with that the **Numpy** Python Library and the Matplotlib graphical representation Library is also used.

02 The Fashion MNIST dataset has **60,000 training images** and **10,000 testing images**. Both the test and training images are reshaped into a 4-D Tensor where each image is **28 X 28 pixels and in Grayscale format** using the **reshape** function. To scale the pixel intensities to a range between 0 to 1, each of them is divided by 255(maximum value a pixel can attain) and normalized. Normalizing the input data helps in reducing the impact of different scales and ranges of pixel intensities among images, making the optimization process more stable and efficient.



MY APPROACH TOWARDS THE PROBLEM(contd..)

03

Three convolutional layers are created using **Conv2D** class provided by the Keras and Tensorflow library. The first layer has 32 filters followed by 64 and 128 filters by the second and third layers respectively. The size of the Convolution Kernel is 3 X 3 which slides over the input data and performs convolution operation. The **Rectified Linear Unit (ReLU)** activation function is used, which introduces non-linearity into the network.

04

Three **Max Pooling layers** are added after the convolutional layers. The max pooling operation divides the input feature map into non-overlapping rectangular regions of this size and outputs the maximum value within each region. By adding these max pooling layers to the model architecture, the feature maps obtained from the convolutional layers will be downsampled, progressively reducing their spatial dimensions.

MY APPROACH TOWARDS THE PROBLEM(contd..)

05

The layers are flattened using the **Flatten** function to reshape the input data from a multi-dimensional tensor to a one-dimensional vector. This is often required when transitioning from convolutional layers to fully connected layers. Just after the flattening the **Dense layer**, which is a **fully connected layer** is integrated. It connects every neuron from the previous layer to every neuron in the current layer.
Next the output layer is added with 10 neuron units and the Softmax activation function. The softmax function is often used in multi-class classification problems as it normalizes the output values into a probability distribution over the different classes. The output layer provides the final predictions of the model, often with the softmax activation function to produce class probabilities.

MY APPROACH TOWARDS THE PROBLEM(contd..)

06

The model is then compiled. It is a crucial step in preparing the model for training. It configures the model with the specified optimizer, loss function, and metrics. This step allows the model to be ready for the subsequent training process, where it learns from the training data and adjusts its weights and biases to minimize the loss and improve the specified metrics.

The **optimizer parameter** specifies the optimization algorithm to be used during the training process. In this case, the Adam optimizer is chosen. **Adam (short for Adaptive Moment Estimation)** is a popular optimization algorithm that combines the advantages of both AdaGrad and RMSProp. It is well-suited for training deep neural networks and performs automatic learning rate adaptation.

The **loss parameter** determines the loss function that is used to compute the model's error during training. In this case, the **sparse categorical cross-entropy loss function** is selected. This loss function is commonly used for multi-class classification tasks where the target labels are integers (as opposed to one-hot encoded vectors). It calculates the cross-entropy loss between the predicted class probabilities and the true class labels.

The **metrics parameter** specifies the evaluation metrics to be used to monitor the model's performance during training and evaluation. In this case, the chosen metric is accuracy. **Accuracy** measures the percentage of correctly predicted labels out of the total number of samples. It provides a measure of how well the model is classifying the data.

MY APPROACH TOWARDS THE PROBLEM(contd..)

07 Using the **fit function**, the model is trained. It takes the training data and labels as input, along with various configuration parameters. During the training process, the model will iteratively update its weights and biases based on the provided data and loss function. The validation split allows for monitoring the model's performance on unseen data, and the **number of epochs** controls the overall training duration.
The **validation split** specifies the proportion of the training data that will be used for validation during training.
The **batch size** determines the number of samples per batch that will be used for each gradient update. The training data will be divided into batches of **size 512**, and the model's weights will be updated after processing each batch

08 After training the model the **EarlyStopping** function is called to prevent overfitting. Followed by this the model is evaluated upon the testing images, where the model shows and **accuracy of around 92.36%** using the Intel Optimization for Tensorflow. Using the **time library** of Python the **baseline inference latency for 1000 images** is calculated which comes out to be **0.0764 seconds per image**.

Results with and without INTEL OPTIMIZATION FOR TENSORFLOW

with optimization

TF_ENABLE_ONEDNN_OPTS=1

Training Accuracy : **0.9961**
Evaluation Accuracy : **0.9213**

Loss : **0.4807**

Baseline Inference Latency : **0.0753** seconds

without optimization

TF_ENABLE_ONEDNN_OPTS=0

Training Accuracy : **0.9966**
Evaluation Accuracy : **0.9172**

Loss : **0.4465**

Baseline Inference Latency : **0.0780** seconds

Link to the Github Repository

https://github.com/Piyush-18/Intel_Unnati_Project

THANKS

REFERENCES

01 https://www.mathworks.com/discovery/deep-learning.html#:~:text=Deep%20learning%20is%20a%20machine,a%20pedestrian%20from%20a%20lamppost.

02 https://towardsdatascience.com/convolutional-neural-networks-explained-9cc5188c4939

03 https://www.mdpi.com/2073-8994/14/7/1325

04 https://www.ibm.com/topics/convolutional-neural-networks

05 https://www.activestate.com/resources/quick-reads/what-is-a-keras-model/

06 https://www.kaggle.com/datasets/zalando-research/fashionmnist